

QUIZ

Corporación Universitaria del Huila – CORHUILA

Asignatura: Arquitectura de Software

Maria Fernanda Robayo Laguna

Clonación exitosa del repositorio del sistema.

```
C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [Version 10.0.26100.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\robay\OneDrive\Documentos\Quiz>git clone https://github.com/lanvargas94/Semana3_quiz
Cloning into 'Semana3_quiz'...
remote: Enumerating objects: 27, done.
remote: Counting objects: 100% (27/27), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 27 (delta 0), reused 27 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (27/27), 4.29 KiB | 399.00 KiB/s, done.

C:\Users\robay\OneDrive\Documentos\Quiz>
```

Proceso de construcción de imágenes Docker y despliegue inicial

```
C:\Users\robay\OneDrive\Documentos\Quiz\Semana3_quiz>docker-compose up --build
time="2026-02-23T20:31:09-05:00" level=warning msg="C:\\Users\\robay\\OneDrive\\Documentos\\Quiz\\Semana3_quiz\\docker-c
ompose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 577B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile:
#2 transferring dockerfile: 316B 0.0s done
#2 DONE 0.2s

#3 [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-17
#3 ...

#4 [auth] library/eclipse-temurin:pull token for registry-1.docker.io
#4 DONE 0.0s

#5 [auth] library/maven:pull token for registry-1.docker.io
#5 DONE 0.0s

#6 [internal] load metadata for docker.io/library/eclipse-temurin:17-jre
#6 DONE 2.4s

#3 [internal] load metadata for docker.io/library/maven:3.9-eclipse-temurin-17
#3 DONE 2.4s
```

Verificación de contenedores activos y puertos expuestos.

FASE 2 — Auditoría del código

Tabla de Hallazgos — AuthController.java

#	Descripción del problema	Archivo	Línea aprox.	Principio violado	Riesgo
1	Uso de nombres de parámetros poco descriptivos (<code>u</code> , <code>p</code> , <code>e</code>) que afectan legibilidad y claridad del código	AuthController.java	20–28	Clean Code (Naming claro y descriptivo)	Bajo
2	Uso de <code>throws Exception</code> genérico en los endpoints, sin manejo específico de errores	AuthController.java	20, 26	Clean Code (Manejo adecuado de excepciones)	Medio
3	Retorno de <code>Map<String, Object></code> en lugar de un DTO tipado, lo que debilita el contrato de la API	AuthController.java	19–29	Clean Architecture / Buen diseño de API	Medio
4	Ausencia de validación de parámetros de entrada (<code>@Valid</code> , <code>@NotBlank</code> , <code>@Email</code> , etc.) en el controlador	AuthController.java	20–28	Seguridad básica / Separación de responsabilidades	Medio

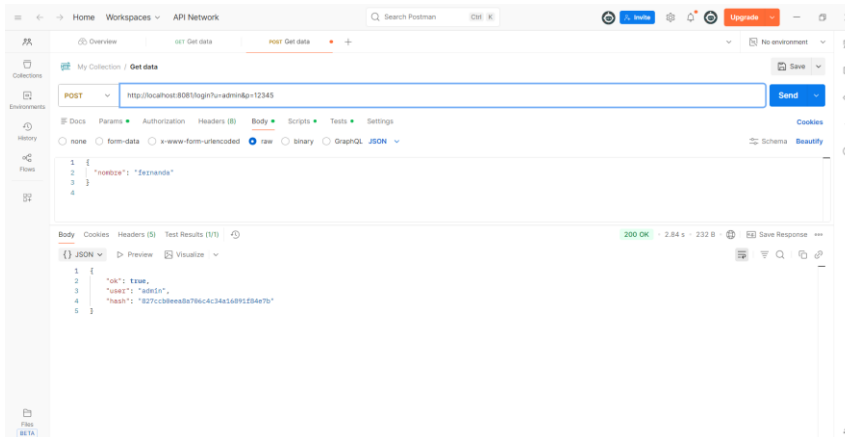
 Tabla de Hallazgos — AuthService.java

Lista para copiar y pegar en Word (formato formal)

#	Descripción del problema	Archivo	Línea aprox.	Principio violado	Riesgo	
1	Uso de algoritmo MD5 para hash de contraseñas (inseguro y obsoleto)	AuthService.java	56–67	Seguridad básica (Hashing inseguro)	Alto	
2	Retorno del hash de la contraseña en la respuesta del login	AuthService.java	27, 32	Exposición de información sensible	Alto	
3	Acceso directo a atributos públicos (<code>c.password</code> , <code>c.email</code> , <code>c.username</code>)	AuthService.java	23–28, 44–46	Encapsulamiento (Clean Code / OOP)	Medio	
4	Logging mediante <code>System.out.println</code> exponiendo información sensible	AuthService.java	24–31, 47–51	Seguridad / Buenas prácticas de logging	Alto	

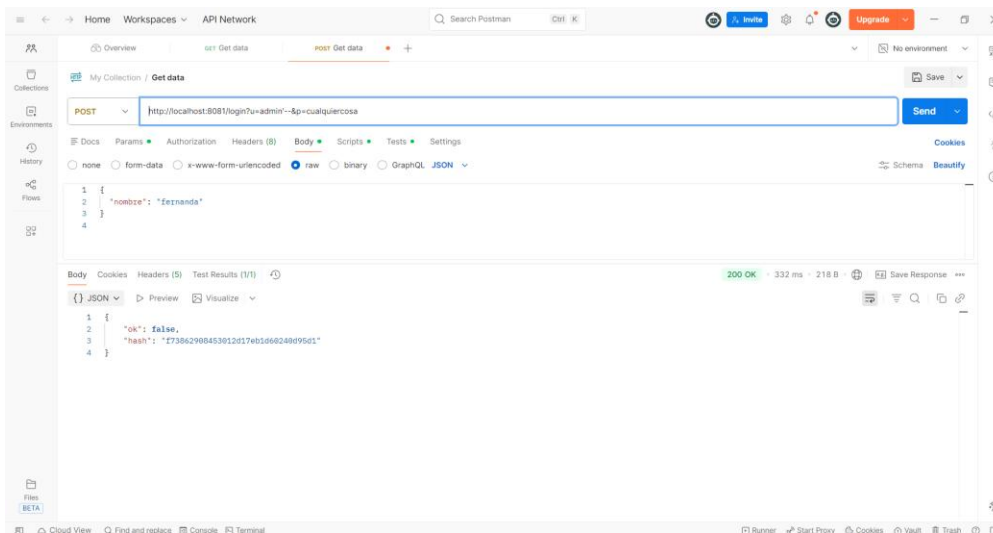
FASE 3 — Pruebas funcionales

Resultado de la Prueba 1 — Login válido



El endpoint funciona funcionalmente, pero presenta una vulnerabilidad crítica al exponer el hash de la contraseña en la respuesta.

Prueba 2 — SQL Injection



La prueba de inyección SQL no logró comprometer la autenticación.

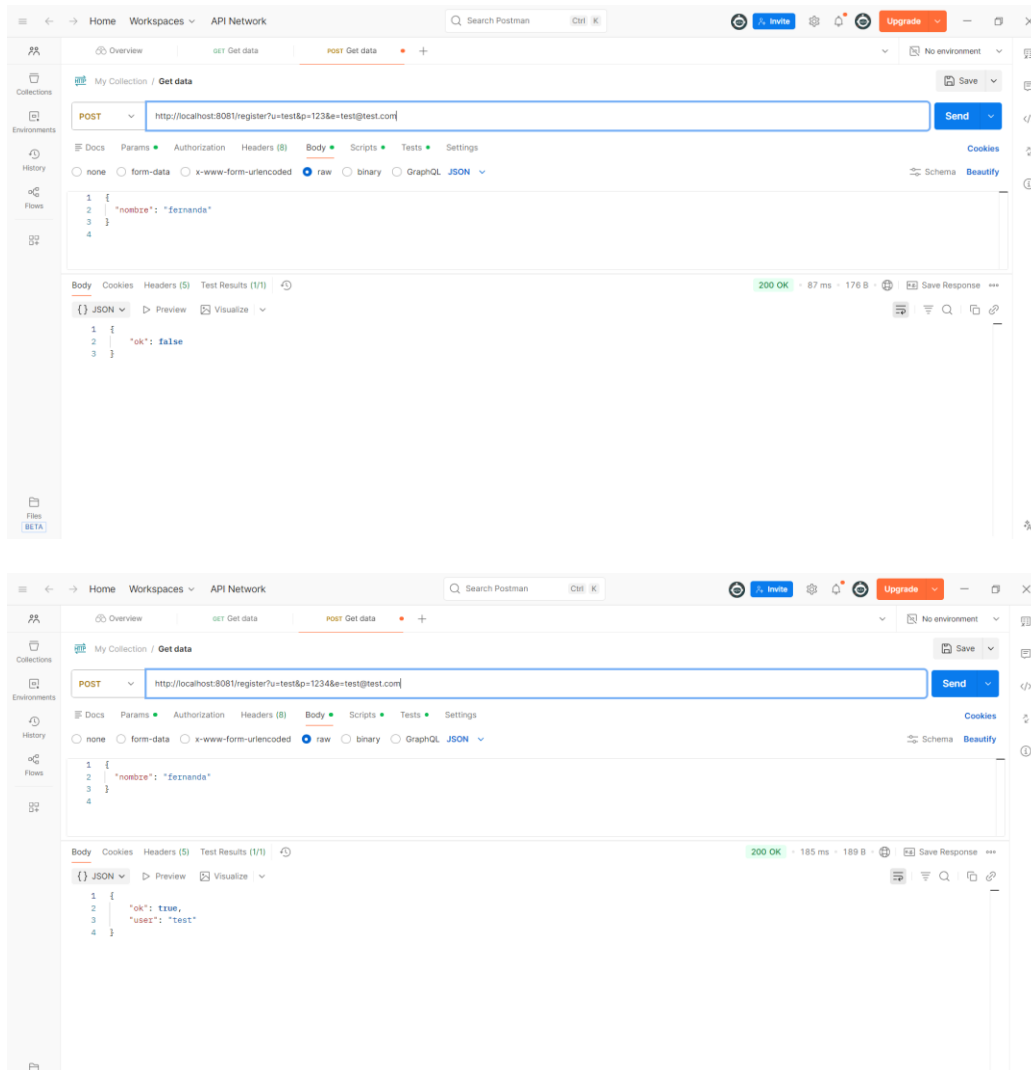
Sin embargo, el endpoint continúa exponiendo el hash de la contraseña incluso en intentos fallidos, lo cual constituye una vulnerabilidad de seguridad.

Nivel de riesgo detectado en esta prueba:

- SQL Injection: No evidenciado (requiere revisión del repositorio)

- Exposición de hash: Alto

Prueba 3 — Registro con contraseña débil



El sistema rechaza contraseñas de menos de 4 caracteres, pero acepta contraseñas débiles como "1234".

La validación es insuficiente para un entorno productivo y no cumple estándares básicos de seguridad actuales.

Nivel de riesgo: **Medio**

ADR-001: Refactorización del módulo de autenticación por vulnerabilidades de seguridad y malas prácticas de Diseño

El sistema actual permite registro y autenticación de usuarios mediante un módulo compuesto por controller, service y repository. Durante la auditoría se identificaron problemas críticos: uso de MD5 para contraseñas, exposición del hash en las respuestas del login, validación débil de contraseñas y logging inseguro con `System.out.println`. Además, se detectaron violaciones al principio de responsabilidad única y uso de estructuras genéricas en las respuestas.

Estas fallas representan un riesgo alto en producción, ya que exponen información sensible y facilitan ataques de fuerza bruta. Esto afecta directamente a los usuarios (seguridad), al equipo técnico (mantenibilidad) y al negocio (riesgo reputacional).

Decisión

1. Se reemplazará MD5 por BCrypt para el almacenamiento seguro de contraseñas.
2. Se eliminará el campo hash de las respuestas y se implementarán DTOs tipados.
3. Se reforzará la validación de contraseñas (mínimo 8 caracteres y mayor complejidad).
4. Se reemplazará `System.out.println` por logging estructurado.
5. Se refactorizará el servicio aplicando SRP para separar validación, autenticación y construcción de respuesta.

La arquitectura quedará con responsabilidades claras: Controller (entrada), Service (lógica), Repository (persistencia) y DTOs (contratos).

Consecuencias

Positivas

- Mayor seguridad.
- Eliminación de exposición de datos sensibles.

- Código más limpio y mantenible.
- Mejor cumplimiento de buenas prácticas.

Riesgos

- Tiempo adicional de refactorización.
- Posibles regresiones si no se agregan pruebas.
- Ajustes necesarios en contratos de API.

Alternativas consideradas

1. Reescribir el módulo completo → descartado por alto costo y tiempo innecesario.
2. Corregir solo el hashing sin refactorizar estructura → descartado porque no solucionaría problemas de diseño ni exposición de datos.